

Radix-2

DP8 (8×4) — one digit per bit of B

Step 1 — the dot product

$$\sum_{i=0}^7 A_i B_i$$

Step 2 — expand B into its bits and exchange the sums

Each B_i is a 4-bit value read as signed: the top bit counts negative, $B_i = \sum_{j=0}^3 w_j B_i^j$ with $w = (1, 2, 4, -8)$, so

$$\sum_{i=0}^7 A_i B_i = \sum_{i=0}^7 \left[\sum_{j=0}^3 w_j B_i^j \right] A_i = \sum_{j=0}^3 w_j \sum_{i=0}^7 B_i^j A_i$$

When B_i is a lower slice of a wider B the top bit is read unsigned, $w_3 = +8$.

Step 3 — the partial product

$$\sum_{i=0}^7 A_i B_i = \sum_{j=0}^3 w_j \sum_{i=0}^7 \underbrace{[B_i^j A_i]}_{R(B_i^j)}$$

$R(B_i^j)$ depends on one bit and takes one of two values; the sign never enters the digit, it is the weight of the top plane, so exactly one row is subtracted:

B_i^j	d_i^j	R
0	0	0
1	1	A_i

Step 4 — split the work

There is nothing to encode: the digit is the bit itself, and the dispatcher sends it to the PE as the control signal of the selection; only the selection stays inside:

$$\underbrace{\{B_i^j\} \longrightarrow d_i^j}_{\text{dispatcher: encoding}} \longrightarrow \underbrace{\sum_{j=0}^3 w_j \sum_{i=0}^7 d_i^j A_i}_{\text{PE: selection}}$$

Each d_i^j travels as its bit, which the cell turns into one of $\{0, A_i\}$: 4 control bits per lane.

B is the narrow operand: 4 digits × 8 lanes = 32 selections of 8 bits per DP8, one multiple (A_i) and no encoder, at twice the rows of every radix-4 scheme.